

- 在立即数寻址中，**操作数本身直接在指令中**给出，取出指令也就获得了操作数，这个操作数也称为立即数。
- # 后的立即数形式：
 - 0x 或 &** 表示十六进制数
 - 0b** 表示二进制数
 - 0d** 或缺省表示十进制数
- 例：

```
ADD R0, R1, # 5      ; R0=R1 + 5
MOV R0, # 0x55        ; R0=0x55
```
- 其中：操作数 **5**，**0x55** 就是立即数，立即数在指令中要以“#”为前缀，后面跟实际数值。

- 将寄存器（称为基址寄存器）的值与指令中给出的偏移地址量相加，**所得结果作为操作数的物理地址。**
- 变址寻址方式常用于访问某基地址附近的地址单元。

- 例：

```
LDR R0 , [R1 , # 5]      ; R0=[R1+5]
LDR R0 , [R1 , R2]       ; R0=[R1+R2]
```

- **寻址方式**：就是根据指令中操作数的信息寻找操作数实际物理地址的方式。
- 依据指令中给出的操作数的不同格式，**ARM** 指令系统具有 **8 种**常见的寻址方式：
 - ✓ 寄存器寻址
 - ✓ 立即寻址
 - ✓ 寄存器间接寻址
 - ✓ 变址寻址
 - ✓ 寄存器移位寻址
 - ✓ 多寄存器寻址
 - ✓ 堆栈寻址
 - ✓ 相对寻址

- **合理常数**：一个 32 位数用 12 位编码表示，符合以下规则才是合法常数。
常数 = imm8 循环右移 ($2 \times \text{rotate_imm}$)

- 例如：

- 汇编指令 `mov r0,#0x0000f200`

- 机器指令 0xe3a0cf2，其中 0xcf2 为立即数。

- **Immed_8=0xf2 rotate_imm=0x0c**

- 常数 0xf2 (循环右移动 24 位)

0x0000f200= 0xf2 循环右移 (2×0x0c)

11-8	7-0
循环部分	立即数部分
Rotate_imm	Immed_8

31-28		27-25				24-21				20				19-16				15-12				11-0			
cond		opcode								s				Rn				Rd				Op2			
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20				
移动次数																									
每次移动 2 位		3130	2928	2726	2524	2322	2120	1918	1716	1514	1312	1110	98	76	54	32	10								
	第 1 次	10	00	00	00	00	00	00	00	00	00	00	00	00	11	11	00	10							
	第 2 次	00	10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	11	11	00					
	第 3 次	11	00	10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	11					
	第 12 次	00	00	00	00	00	00	00	00	00	11	11	00	10	00	00	00	00	00	00					

3.1.5 寄存器移位寻址

- 寄存器移位寻址是 ARM 指令集**独有的**寻址方式。
- 寄存器移位寻址的操作数由**寄存器的数值做相应移位**而得到。
- 移位的方式在指令中以助记符的形式给出，而移位的位数可用立即数或寄存器寻址方式表示。
- ARM 微处理器内嵌桶型移位器，移位操作在 ARM 指令集中不作为单独的指令使用，它只能作为指令格式中的一个字段，在汇编语言中表示为指令中的选项。

• 例：

ADD R0 , R1 , R2 , ROR # 5
; R0=R1 + R2 循环右移 5 位
MOV R0 , R1 , LSL R3
; R0=R1 逻辑左移 R3 位

- ARM 指令集共有 5 种位移操作。

3.1.1 寄存器寻址

- 在寄存器寻址方式下，寄存器的值即为操作数。
- ARM 指令普遍采用此种寻址方式。

- 例：

```
ADD    R0 , R1 , R2      ; R0=R1 + R2
MOV    R0 , R1           ; R0=R1
```

- 寄存器中的值是操作数的物理地址。
- 而实际的操作数存放在存储器中。

- 例：

```
STR R0 , [R1] ; [R1]=R0
LDR R0 , [R1] ; R0=[R1]
```

- 格式为：

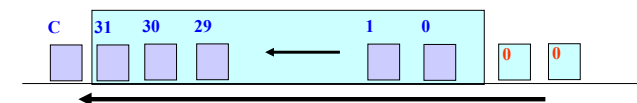
通用寄存器, **LSL** (或 **ASL**) 操作数

- **功能：**LSL（或 ASL）可完成对通用寄存器中的内容进行**逻辑（或算术）的左移**操作，按操作数所指定的数量**向左移位**，低位用零来填充，最后一个左移出的位放在状态寄存器的 C 位。

- **操作数：**通用寄存器，也可以是立即数（0 ~ 31）。

- 操作示例：

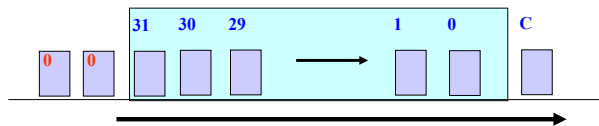
MOV R0, R1, LSL #2



LSR 逻辑右移

- 格式为：
通用寄存器，LSR 操作数
- 功能：LSR 可完成对通用寄存器中的内容进行逻辑右移的操作，按操作数所指定的数量向右移位，左端用零来填充，最后一个右移出的位放在状态寄存器的 C 位。
- 操作数：通用寄存器，也可以是立即数（0 ~ 31）。
- 操作示例：

MOV R0, R1, LSR #4

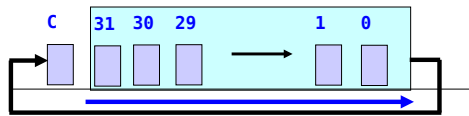


Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 10

RRX 带扩展的循环右移

- 格式为：
通用寄存器，RRX
- 功能：RRX 可完成对通用寄存器中的内容进行带扩展的循环右移的操作，向右循环移动 1 位，左侧空位由状态寄存器 C 位来填充，右侧移出的位移进状态位 C 中。
- 注意：没有操作数，每次执行指令只能移动一位。
- 操作示例：

MOV R0, R1, RRX



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 13

3.1.8 相对寻址

- 相对寻址同基址变址寻址相似，区别只是将程序计数器 PC 作为基址寄存器，指令中的标记作为地址偏移量。
- 例：

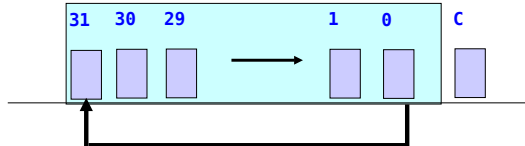
```
BEQ process1
.....
process1
.....
```

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 16

ROR 循环右移

- 格式为：
通用寄存器，ROR 操作数
- 功能：ROR 可完成对通用寄存器中的内容进行循环右移的操作，按操作数所指定的数量向右循环移位，右端移出的位填充在左侧的空位处，最后一个右移出的位同时也放在状态寄存器的 C 位。
- 操作数：通用寄存器，也可以是立即数（0 ~ 31）。
- 操作示例：

MOV R0, R1, ROR #4



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 11

3.1.6 多寄存器寻址

- 在多寄存器寻址方式中，一条指令可实现一组寄存器值的传送。
- 这种寻址方式可以一次对多个寄存器寻址，多个寄存器由小到大批列，最多可传送 16 个寄存器。
- 连续的寄存器间用“-”连接，否则用“，”分隔。
- 例：
LDMIA R0, {R1-R5} ; R1=[R0]
; R2=[R0+4]
; R3=[R0+8]
; R4=[R0+12]
; R5=[R0+16]
指令中 IA 表示在执行完一次 Load 操作后，R0 自增 4。
该指令将以 R0 为起始地址的 5 个字数据分别装入 R1, R2, R3, R4, R5 中。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 14

3.2 ARM 指令集

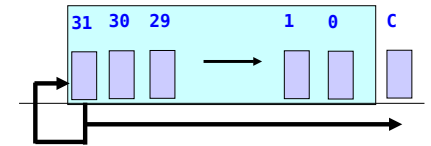
- 机器指令：CPU 指令，能被处理器直接执行，而伪指令宏和宏指令不能。机器指令包括 ARM 指令集和 Thumb 指令集；
- 伪指令：汇编指令，在源程序汇编期间，由汇编编译器处理。其作用是为汇编程序完成准备工作；
- 宏指令：汇编指令，在程序中用于调用宏，宏是一段独立的程序代码；在程序汇编时，对宏调用进行展开，用宏体代替宏指令。
- ARM 微处理器的指令集是加载/存储型的，即指令集仅能处理寄存器中的数据，处理结果仍要放回寄存器中，而对系统存储器的访问则需要通过专门的加载/存储指令来完成。
- ARM9 指令集，包括 ARM 指令集、Thumb 指令集。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 17

ASR 算术右移

- 格式为：
通用寄存器，ASR 操作数
- 功能：ASR 可完成对通用寄存器中的内容进行算术右移的操作，按操作数所指定的数量向右移位，最左端的位保持不变，最后一个右移出的位放在状态寄存器的 C 位。
- 操作数：通用寄存器，也可以是立即数（0 ~ 31）。
- 操作示例：

MOV R0, R1, ASR #4



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 12

3.1.7 堆栈寻址

- 堆栈：按照“先进后出”的原则组织的特定的数据存储的区域。
- 使用专门的寄存器（堆栈指针 SP(R13)）指向堆栈栈顶。
- 堆栈寻址用于数据栈与寄存器组之间批量数据传输。
- 当数据写入和读出内存的顺序不同时，使用堆栈寻址可以很好的解决这个问题。
- 例：
STMFD R13!, {R0,R1,R2,R3,R4}
; 将 R0-R4 中的数据压入堆栈，R13 为堆栈指针
LDMFD R13!, {R0,R1,R2,R3,R4}
; 将数据出栈，恢复 R0-R4 原先的值

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 15

3.2.1 指令格式

- 实际指令语法格式为：

ADDEQS R0, R1, R2

- 该指令的编码格式为：

31 ~ 28	27 ~ 25	24 ~ 21	20	19 ~ 16	15 ~ 12	11 ~ 0
cond		opcode	S	Rn	Rd	op2
0000	001	0100	1	0001	0000	000000000010

注：该指令格式仅针对本例的指令，不同的指令有不同的指令格式。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 18

		31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0																															
Data processing immediate shift		cond	[1]	0	0	0	opcode	S	Rn	Rd	shift amount	shift	0	Rm																			
Miscellaneous instructions: See Figure 3-3		cond	[1]	0	0	0	1	0	x	0	x	x	x	x	x	x	x	x	x	x	x	x	x	0	x	x	x	x	0	x	x	x	
Data processing register shift [2]		cond	[1]	0	0	0	opcode	S	Rn	Rd	Rs	0	shift	1	Rm																		
Miscellaneous instructions: See Figure 3-3		cond	[1]	0	0	0	1	0	x	0	x	x	x	x	x	x	x	x	x	x	x	0	x	1	x	x	x	0	x	x	x		
Multiplies, extra loadstores: See Figure 3-2		cond	[1]	0	0	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	1	x	1	x	x	x	0	x	x	x			
Data processing immediate [2]		cond	[1]	0	0	1	opcode	S	Rn	Rd	rotate	immediate																					
Undefined instruction [3]		cond	[1]	0	0	1	0	x	0	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x		
Move immediate to status register		cond	[1]	0	0	1	0	R	1	0	Mask	SBO	rotate	immediate																			
Loadstore immediate offset		cond	[1]	0	1	0	P	U	B	W	L	Rn	Rd	immediate																			
Loadstore register offset		cond	[1]	0	1	0	P	U	B	W	L	Rn	Rd	shift amount	shift	0	Rm																
Undefined instruction		cond	[1]	0	1	1	x	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	1	x	x	x	0	x	x	x		
Undefined instruction [4,7]		1	1	1	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x		
Loadstore multiple		cond	[1]	1	0	0	P	U	S	W	L	Rn					register list																
Undefined instruction [4]		1	1	1	1	0	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x		
Branch and branch with link		cond	[1]	1	0	1	L						24-bit offset																				
Branch and branch with link and change to Thumb [4]		1	1	1	1	0	1	H						24-bit offset																			
Coprocessor loadstore and double register transfer [5]		cond	[5]	1	1	0	P	U	N	W	L	Rn	CRd				cp_num	8-bit offset															
Coprocessor data processing		cond	[5]	1	1	1	0	opcode1	CRn	CRd				cp_num	opcode2	0	CRm																
Coprocessor register transfers		cond	[5]	1	1	1	0	opcode1	L	CRn	Rd				cp_num	opcode2	1	CRm															
Software interrupt		cond	[1]	1	1	1	1	swi number																									
Undefined instruction [4]		1	1	1	1	1	1	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x		

3.2.2 条件码

- 几乎所有的 ARM 指令都可以根据当前程序状态寄存器 CPSR 中标志位的值, 有条件地执行。
- ARM 指令的条件域 <cond> 有 16 种类型。

	cond	编码	CPSR 中标志位	含 义
相等	EQ	0000	Z 置位	相等
	NE	0001	Z 清零	不相等
负数	MI	0100	N 置位	负数
	PL	0101	N 清零	正数或零
溢出	VS	0110	V 置位	溢出
	VC	0111	V 清零	未溢出
无符号数	CS/HS	0010	C 置位	无符号数大于或等于
	CS/O	0011	C 清零	无符号数小于
	HI	1000	C 置位 Z 清零	无符号数大于
	LS	1001	C 清零 Z 置位	无符号数小于或等于
有符号数	GE	1010	N 等于 V	带符号数大于或等于
	LT	1011	N 不等于 V	带符号数小于
	GT	1100	Z 清零且 (N 等于 V)	带符号数大于
	LE	1101	Z 置位或 (N 不等于 V)	带符号数小于或等于
	AL	1110	忽略	无条件执行
		1111	依版本不同, 定义不同	

3.2.4 ARM 数据处理类指令

- 数据处理指令 **只能对寄存器的内容进行操作**，不允许对存储器中的数据进行操作，也不允许指令直接使用存储器的数据或在寄存器与存储器之间传送数据。
- 数据处理指令可分为 **3 大类**：**数据传送指令**、**算术逻辑运算指令**、**比较指令**。
- **数据传送指令**：用于在寄存器和存储器之间进行数据的双向传输。
- **算术逻辑运算指令**：完成常用的算术与逻辑的运算，该类指令不但将运算结果保存在目的寄存器中，同时更新 **CPSR** 中的相应条件标志位。
- **比较指令**：完成对指定的两个寄存器（或 **1 个寄存器**，**1 个立即数**）进行比较，不保存运算结果，只影响 **CPSR** 中相应的条件标志位。

ARM 指令的助记符

- ARM 指令在汇编程序中用助记符表示，一般 ARM 指令的助记符格式为：

`<opcode> {<cond>} {S} <Rd>,<Rn>,<op2>`
- 其中：
 - < > 是必选项
 - { } 是可选项
 - <opcode> 操作码，如 ADD 表示算术加操作指令；
 - {<cond>} 决定指令执行的条件域；
 - {S} 决定指令执行是否影响 CPSR 寄存器的值；
 - <Rd> 目的寄存器；
 - <Rn> 第一个操作数，为寄存器；
 - <op2> 第二个操作数。
- 例如，指令 `ADDEQS R1, R2, #5`

条件助记符 -- 英文解释

Opcode [31:28]	Mnemonic extension	Meaning	Condition flag state
0000	EQ	<u>E</u> qual	Z set
0001	NE	<u>N</u> ot <u>e</u> qual	Z clear
0010	CS/HS	<u>C</u> arry <u>s</u> et/ <u>u</u> nsigned <u>h</u> igher or <u>s</u> ame	C set
0011	CC/LO	<u>C</u> arry <u>c</u> lear/ <u>u</u> nsigned <u>l</u> ower	C clear
0100	MI	<u>M</u> inus/ <u>n</u> egative	N set
0101	PL	<u>P</u> lus/ <u>p</u> ositive or <u>z</u> ero	N clear
0110	VS	<u>O</u> verflow	<u>V</u> set
0111	VC	<u>N</u> o overflow	<u>V</u> <u>c</u> lear
1000	HI	<u>U</u> nsigned <u>h</u> igher	C set and Z clear
1001	LS	<u>U</u> nsigned <u>l</u> ower or <u>s</u> ame	C clear or Z set
1010	GE	<u>S</u> igned <u>g</u> reater than or <u>e</u> qual	N set and V set, or N clear and V clear (N == V)
1011	LT	<u>S</u> igned <u>l</u> ess <u>t</u> han	N set and V clear, or N clear and V set (N != V)
1100	GT	<u>S</u> igned <u>g</u> reater <u>t</u> han	Z clear, and either N set and V set, or N clear and V clear (Z == 0, N == V)
1101	LE	<u>S</u> igned <u>l</u> ess than or <u>e</u> qual	Z set, or N set and V clear, or N clear and V set (Z == 1 or N != V)
1110	AL	<u>A</u> lways (unconditional)	-

MOV 数据传送指令

- **格式：**MOV{<cond>}[S] <Rd>, <op1>
 - **功能：**Rd ← op1
 - **Op1：**寄存器、被移位的寄存器、立即数。
-
- **例如：**
 - MOV R0, # 5 ; R0=5
 - MOV R0, R1 ; R0=R1
 - MOV R0, R1, LSL # 5 ; R0=R1 左移

第 2 个操作数

- ARM 指令在汇编程序中用助记符表示，一般 ARM 指令的助记符格式为：

$$\text{<opcode> \{<cond>\} \{S\} <Rd>, <Rn>, <op2>}$$
- 灵活地使用第 2 个操作数“op2”能够提高代码效率。它有如下的形式：
 - #immed_8r：常数表达式；
 - Rm：寄存器方式；
 - Rm, shift：寄存器移位方式。

条件 - 举例

- **CMP** R0, R1 ; R0 与 R1 比较
- **ADDHI** R0, R0, #1 ; 若 $R0 > R1$, 则 $R0 = R0 + 1$
- **ADDLS** R1, R1, #1 ; 若 $R0 \leq R1$, 则 $R1 = R1 + 1$

	cond	编码	CPSR 中标志位	含 义
相等	EQ	0000	Z 置位	相等
	NE	0001	Z 清零	不相等
失败	MI	0100	N 置位	失败
	PL	0101	N 清零	正数或零
溢出	VS	0110	V 置位	溢出
	VC	0111	V 清零	未溢出
无符号数	CS/HS	0010	C 置位	无符号数大于或等于
	CC/LO	0011	C 清零	无符号数小于
	HI	1000	C 置位 Z 清零	无符号数大于
	LS	1001	C 清零 Z 置位	无符号数小于或等于
有符号数	GE	1010	N 等于 V	带符号数大于或等于
	LT	1011	N 不等于 V	带符号数小于
	GT	1100	Z 清零且 (N 等于 V)	带符号数大于
	LE	1101	Z 置位或 (N 不等于 V)	带符号数小于或等于
	AL	1110	忽略	无条件执行
		1111	依版本不同, 定义不同	

MVN 数据取反传送指令

- **格式：** MVN{<cond>}{S} <Rd>, <op1>
- **功能：** op1 **按位取反**→ Rd , 即 Rd=!op1 。
- Op1 : 寄存器、被移位的寄存器、立即数。
- **例如：**
- MVN R0, # 0 ; R0=-1

ADD 加法指令

- 格式：ADD{<cond>}{S} <Rd>, <Rn>, <op2>
- 功能：Rd ← Rn + op2
- Op2：寄存器、被移位的寄存器、立即数。

- 例如：
 - ADD R0, R1, # 5 ; R0=R1+5
 - ADD R0, R1, R2 ; R0=R1+R2
 - ADD R0, R1, R2, LSL # 5 ; R0=R1+R2 左移 5 位

SBC 带借位减法指令

- 格式：SBC{<cond>}{S} <Rd>, <Rn>, <op2>
- 功能：Rd ← Rn - op2 - !carry
- Op2：寄存器、被移位的寄存器、立即数。
- SUB 和 SBC 生成进位标志的方式不同于常规，如果需要借位则清除进位标志，所以指令要对进位标志进行一个非操作。
- 例如：
 - 第一个 64 位操作数存放在寄存器 R3 R2 中；
 - 第二个 64 位操作数存放在寄存器 R5 R4 中；
 - 64 位结果存放在 R1 R0 中。
 - 64 位的减法（第一个操作数减去第二个操作数）可由以下语句实现：
 - SUBS R0, R2, R4 ; 低 32 位相减，S 表示结果影响条件标志位的值
 - SBC R1, R3, R5 ; 高 32 位相减

AND 逻辑与指令

- 格式：AND{<cond>}{S} <Rd>, <Rn>, <op2>
- 功能：Rd ← Rn AND op2
- Op2：寄存器、被移位的寄存器、立即数。
- 一般用于清除 Rn 的特定几位。
- 按位操作。

- 例如：假设 R0=0X12345678
- AND R0, R0, # 5 ; 保持 R0 的第 0 位和第 2 位，其余位清 0

OP1	运算符 (与 ^)	OP2	=	RESULT
0	^	0	=	0
0	^	1	=	0
1	^	0	=	0
1	^	1	=	1

```
0001 0010 0011 0100 0101 0110 0111 1000 0X12345678
^
0000 0000 0000 0000 0000 0000 0000 0101 5
-----
0000 0000 0000 0000 0000 0000 0000 0000
```

ADC 带进位加法指令

- 格式：ADC{<cond>}{S} <Rd>, <Rn>, <op2>
- 功能：Rd ← Rn + op2 + carry
- Op2：寄存器、被移位的寄存器、立即数。
- carry 为进位标志值。
- 该指令用于实现超过 32 位的数的加法。
- 例如：
 - 第一个 64 位操作数存放在寄存器 R3 R2 中；
 - 第二个 64 位操作数存放在寄存器 R5 R4 中；
 - 64 位结果存放在 R1 R0 中。
 - 64 位的加法可由以下语句实现：
 - ADDS R0, R2, R4 ; 低 32 位相加，S 表示结果影响条件标志位的值
 - ADC R1, R3, R5 ; 高 32 位相加

RSB 反向减法指令

- 格式：RSB{<cond>}{S} <Rd>, <Rn>, <op2>
- 功能：同 SUB 指令，但倒换了两操作数的前后位置，即 Rd ← op2 - Rn。
- 例如：
 - RSB R0, R1, # 5 ; R0=5-R1
 - RSB R0, R1, R2 ; R0=R2-R1
 - RSB R0, R1, R2, LSL # 5 ; R0=R2 左移 5 位 -R1

ORR 逻辑或指令

- 格式：ORR{<cond>}{S} <Rd>, <Rn>, <op2>
- 功能：Rd ← Rn OR op2
- Op2：寄存器、被移位的寄存器、立即数。
- 一般用于设置 Rn 的特定几位。

- 例如：假设 R0=0X12345678
- ORR R0, R0, # 5 ; R0 的第 0 位和第 2 位设置为 1，其余位不变

OP1	运算符 (或 V)	OP2	=	RESULT
0	V	0	=	0
0	V	1	=	1
1	V	0	=	1
1	V	1	=	1

```
0001 0010 0011 0100 0101 0110 0111 1000 0X12345678
V
0000 0000 0000 0000 0000 0000 0000 0101 5
-----
0001 0010 0011 0100 0101 0110 0111 1101 0x1234567D
```

SUB 减法指令

- 格式：SUB{<cond>}{S} <Rd>, <Rn>, <op2>
- 功能：Rd ← Rn - op2
- Op2：寄存器、被移位的寄存器、立即数。
- 例如：
 - SUB R0, R1, # 5 ; R0=R1-5
 - SUB R0, R1, R2 ; R0=R1-R2
 - SUB R0, R1, R2, LSL # 5 ; R0=R1-R2 左移 5 位

RSC 带借位的反向减法指令

- 格式：RSC{<cond>}{S} <Rd>, <Rn>, <op2>
- 功能：同 SBC 指令，但倒换了两操作数的前后位置，即 Rd ← op2 - Rn - !carry
- 例如：
 - 第一个 64 位操作数存放在寄存器 R3 R2 中；
 - 第二个 64 位操作数存放在寄存器 R5 R4 中；
 - 64 位结果存放在 R1 R0 中。
 - 64 位的减法（第一个操作数减去第二个操作数）可由以下语句实现：
 - SUBS R0, R4, R2 ; 低 32 位相减，S 表示结果影响寄存器 CPSR 的值
 - RSC R1, R5, R3 ; 高 32 位相减

EOR 逻辑异或指令

- 格式：EOR{<cond>}{S} <Rd>, <Rn>, <op2>
- 功能：Rd ← Rn EOR op2
- Op2：寄存器、被移位的寄存器、立即数。
- 一般用于将 Rn 的特定几位取反。

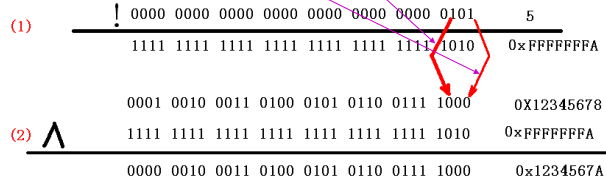
- 例如：假设 R0=0X12345678
- EOR R0, R0, # 5 ; R0 的第 0 位和第 2 位取反，其余位不变

OP1	运算符 (异或 ⊕)	OP2	=	RESULT
0	⊕	0	=	0
0	⊕	1	=	1
1	⊕	0	=	1
1	⊕	1	=	0

```
0001 0010 0011 0100 0101 0110 0111 1000 0X12345678
⊕
0000 0000 0000 0000 0000 0000 0000 0101 5
-----
0001 0010 0011 0100 0101 0110 0111 1101 0x1234567D
```

BIC 位清除指令

- 格式：BIC{<cond>}{S} <Rd>, <Rn>, <op2>
- 功能：Rd ← Rn AND (~op2)
- 用于清除寄存器 Rn 中的某些位，并把结果存放到目的寄存器 Rd 中。
- 操作数 op2 是一个 32 位掩码 (mask)，如果在掩码中设置了某一位，则清除 Rn 中的这一位；未设置的掩码位指示 Rn 中此位保持不变。
- Op2：寄存器、被移位的寄存器、立即数。
- 例如：假设 R0=0X12345678
- BIC R0, R0, #5 ; R0 中第 0 位和第 2 位清 0，其余位不变



SMULL 64 位有符号数乘法指令

- 格式：
SMULL{<cond>}{S} <Rdl>, <Rdh>, <Rn>, <op2>
- 功能：Rdh Rdl ← Rn × op2
- Rdh、Rdl、op2：均为寄存器。
- Rn 和 op2 的值为 32 位的有符号数。
- 例如：
SMULL R0, R1, R2, R3 ; R0 = R2 × R3 的低 32 位
; R1 = R2 × R3 的高 32 位

UMLAL 64 位无符号数乘加指令

- 格式：
UMLAL{<cond>}{S} <Rdl>, <Rdh>, <Rn>, <op2>;
- 功能：同 SMLAL 指令：
Rdh Rdl ← Rn × op2 + Rdh Rdl
- 但 Rn，op2 的值为 32 位的无符号数，Rdh Rdl 的值为 64 位无符号数。
- 例如：
UMLAL R0, R1, R2, R3 ; R0 = R2 × R3 的低 32 位 + R0
; R1 = R2 × R3 的高 32 位 + R1
- 其中 R2、R3 的值为 32 位无符号数
- R1、R0 的值为 64 位无符号数

MUL 32 位乘法指令

- 格式：MUL{<cond>}{S} <Rd>, <Rn>, <op2>
- 功能：Rd ← Rn × op2
- 该指令根据 S 标志，决定操作是否影响 CPSR 的值。
- Op2：必须为寄存器。
- Rn 和 op2 的值为 32 位的有符号数或无符号数。
- 例如：
MULS R0, R1, R2 ; R0 = R1 × R2，结果影响寄存器 CPSR 的值

SMLAL 64 位有符号数乘加指令

- 格式：
SMLAL{<cond>}{S} <Rdl>, <Rdh>, <Rn>, <op2>
- 功能：Rdh Rdl ← Rn × op2 + Rdh Rdl
- Rdh、Rdl、op2：均为寄存器。
- Rn 和 op2 的值为 32 位的有符号数。
- Rdh Rdl 的值为 64 位的加数。
- 例如：
SMLAL R0, R1, R2, R3 ; R0 = R2 × R3 的低 32 位 + R0
; R1 = R2 × R3 的高 32 位 + R1

CMP 比较指令

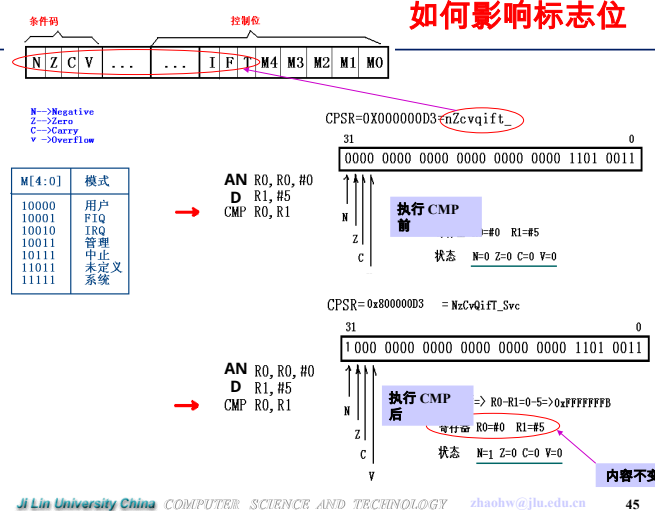
- 格式：CMP{<cond>} <Rn>, <op1>
- 功能：Rn-op1
- 该指令进行一次减法运算，但不存储结果，根据结果更新 CPSR 中条件标志位的值。
- 该指令不需要显式地指定 S 后缀来更改状态标志。
- Op1：寄存器、立即数。
- 例如：
CMP R0, #5 ; 计算 R0-5，根据结果设置条件标志位
- ADDGT R0, R0, #5 ; 如果 R0>5，则执行 ADDGT 指令

MLA 32 位乘加指令

- 格式：MLA{<cond>}{S} <Rd>, <Rn>, <op2>, <op3>
- 功能：Rd ← Rn × op2 + op3
- Op2、op3：必须为寄存器。
- Rn、op2 和 op3 的值为 32 位的有符号数或无符号数。
- 例如：
MLA R0, R1, R2, R3 ; R0 = R1 × R2 + R3

UMULL 64 位无符号数乘法指令

- 格式：
UMULL{<cond>}{S} <Rdl>, <Rdh>, <Rn>, <op2>
- 功能：同 SMULL 指令：
Rdh Rdl ← Rn × op2
- 但 Rn 和 op2 的值为 32 位的无符号数。
- 例如：
UMULL R0, R1, R2, R3 ; R0 = R2 × R3 的低 32 位
; R1 = R2 × R3 的高 32 位
- 其中 R2、R3 的值为无符号数



- 格式：CMN{<cond>} <Rn>, <op1>
- 功能：同 CMP 指令，但寄存器 Rn 的值是和 op1 取反的值进行比较：

Rn - (-op1)

- 例如：
- CMN R0, #5 ; 把 R0 与 -5 进行比较

3.2.5 ARM 分支指令

- 分支指令作用：程序的跳转、程序状态的切换。
- ARM 程序设计中，实现程序跳转有**两种方式**：
 - (1) 跳转指令：跳转空间依据指令确定。
 - (2) 直接向程序寄存器 PC (R15) 中写入目标地址值：在 4GB 空间任意跳转。
- 程序状态切换：ARM 状态、Thumb 状态。

BX 带状态切换的跳转指令

- 格式：BX <Rn>
- 功能：状态切换。
- 处理器跳转到目标地址处，从那里继续执行。
- 目标地址为寄存器 Rn 的值和 0xFFFFFFFF 作与操作的结果。
- 目标地址处的指令可以是 ARM 指令，也可以是 Thumb 指令。

- 例如：
- ADR R0, exit ; 标号 exit 处的地址装入 R0 中
- BX R0 ; 跳转到 exit 处

- 格式：TST{<cond>} <Rn>, <op1>
- 功能：Rn AND op1
- 根据结果更新 CPSR 中条件标志位的值，但不存储结果。
- 用于检查寄存器 Rn 是否设置了 op1 中相应的位。

- 例如：
- TST R0, #5 ; 测试 R0 中第 0 位和第 2 位是否为 1

B 跳转指令 (Branch)

- 格式：B{<cond>} <addr>
- 功能：PC ← PC + addr 左移两位
- 说明：addr 的值是相对当前 PC (即寄存器 R15) 的值的一个偏移量，而不是一个绝对地址，它是 24 位有符号数。实际地址的值由汇编器来计算。
- 目标地址计算方法：
跳转的目的地址 = addr 的值有符号扩展为 32 位后左移两位 (即乘 4) + PC 值
- 跳转的范围为 -32MB ~ +32MB。
- 例如：(类似 8086 JMP)
B exit ; 程序跳转到标号 exit 处
...
exit ...
...
MOV R0, #10 ; 初始化循环计数器
loop ... ; 循环体
SUBS R0, #1 ; 计数器减 1，设置条件码
BNE loop ; 如果计数器 R0 ≠ 0，重复循环

BLX 带返回和状态切换的跳转指令

- 格式：BLX <addr>
BLX <Rn>
- 功能：调用、状态切换。
- 处理器跳转到目标地址处，并将返回地址保存到 LR 寄存器中。
- 如果目标地址处为 Thumb 指令，则程序状态从 ARM 状态切换为 Thumb 状态。
- 例如：
BLX T16 ; 跳转到 T16 处执行，T16 后面的指令为 Thumb 指令
...
CODE16
T16 ; 后面指令为 Thumb 指令
...

- 格式：TEQ{<cond>} <Rn>, <op1>
- 功能：Rn EOR op1
- 将寄存器 Rn 的值和操作数 op1 所表示的值按位作逻辑异或操作，根据结果更新 CPSR 中条件标志位的值，但不存储结果。
- 用于检查寄存器 Rn 的值是否和 op1 所表示的值相等。

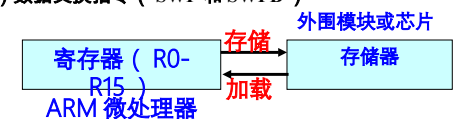
- 例如：
- TEQ R0, #5 ; 判断 R0 的值是否和 5 相等

BL 带返回的跳转指令 (Branch with Link)

- 格式：BL{<cond>} <addr>
- 功能：子程序调用。
- 同 B 指令，但 BL 指令执行跳转操作的同时，还将子程序的返回地址 (注意：不是 PC 内容) 保存到 LR 寄存器 (寄存器 R14) 中。
- 该指令用于实现子程序调用，程序的返回可通过把 LR 寄存器的值复制到 PC 寄存器中来实现。
- 例如：
BL func ; 调用子程序 func
...
func ...
MOV R15, R14 ; 子程序返回
...
CMP R0, #5
BLLT SUB1 ; 若 R0 < 5，调用 SUB1 子程序
BLGE SUB2 ; 否则调用 SUB2 子程序

3.2.3 ARM 存储器访问指令

- 存储器访问指令功能：寄存器和内存之间数据的传送。
- Load (加载)：寄存器 ← 内存
- Store (存储)：内存 ← 寄存器
- 该组指令使用频繁，在指令集中最为重要，因为其他指令只能操作寄存器，当数据存放在内存中时，必须先把数据从内存装载到寄存器，执行完后再把寄存器中的数据存储到内存中。
- Load/Store 指令分为 3 类：
 - (1) 单一数据传送指令 (LDR 和 STR 等)
 - (2) 多数据传送指令 (LDM 和 STM)
 - (3) 数据交换指令 (SWP 和 SWPB)



加载指令与存储指令

加载指令：**LDR** 目标寄存器，源地址

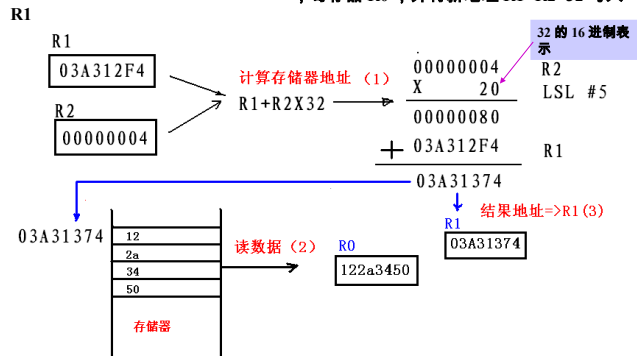


存储指令：**STR** 源寄存器，目标地址



Load/Store 指令过程

- 例如：
- LDR R0, [R1, R2, LSL # 5]!** ; 将内存中地址为 $R1+R2 \times 32$ 的字数据装入寄存器 R0，并将新地址 $R1+R2 \times 32$ 写入 R1



LDRB 字节数据加载指令

- 格式：**LDR{<cond>}B <Rd>, <addr>**
- 功能：同 LDR 指令。但
 - (1) addr 地址内 8 位的字节数据→ Rd
 - (2) Rd 的高 24 位清 0

- 例如：
- LDRB R0, [R1]** ; 将内存中起始地址为 R1 的一个字节数据装入 R0 中

LDR 字数据加载指令

- 格式：**LDR{<cond>} <Rd>, <addr>**
- 功能：
 - (1) addr 所表示的内存地址中的字数据→目标寄存器 Rd
 - (2) 合成的有效地址→基址寄存器。
- 地址 addr：简单的值、偏移量、被移位的偏移量。
- 地址 addr 的寻址方式涉及的寄存器意义：
 - Rn：基址寄存器。
 - Rm：变址寄存器。
 - Index：偏移量，12 位的无符号数。

Load/Store 指令伪代码

```
LDR {<cond>}<Rd>, <addressing_mode>
指令操作的伪代码：
if ConditionPassed(cond) then
  if addresss[1:0] == 0b00 then
    Value = Memory[addresss, 4]
  else if addresss[1:0] == 0b01 then
    Value = Memory[addresss, 4] Rotate_Right 8
  else if addresss[1:0] == 0b10 then
    Value = Memory[addresss, 4] Rotate_Right 16
  else if addresss[1:0] == 0b11 then
    Value = Memory[addresss, 4] Rotate_Right 24
  if (Rd is R15) then
    if (architecture version 5 or above) then
      PC = value AND 0xFFFFFFF
      T Bit = value[0]
    else
      PC = value AND 0xFFFFFFF
  else
    Rd = value
```

LDRH 半字数据加载指令

- 格式：**LDR{<cond>}H <Rd>, <addr>**
- 功能：同 LDR 指令。但
 - (1) addr 地址内 16 位的半字数据→ Rd
 - (2) Rd 的高 16 位清 0
- 例如：
- LDRH R0, [R1]** ; 将内存中起始地址为 R1 的半字数据装入 R0 中

Load/Store 指令一般形式

- LDR Rd, [Rn]** ; 内存地址为 Rn 的字数据→ Rd
- LDR Rd, [Rn, Rm]** ; 内存地址为 Rn+Rm 的字数据→ Rd
- LDR Rd, [Rn, # index]** ; 内存地址为 Rn+index 的字数据→ Rd
- LDR Rd, [Rn, Rm, LSL # 5]** ; 内存地址为 Rn+Rm×32 的字数据→ Rd
- LDR Rd, [Rn, Rm, LSL # 5] !** ; 内存地址为 Rn+Rm 的字数据→ Rd; 并将新地址 Rn+Rm→Rn
- LDR Rd, [Rn, # index] !** ; 内存地址为 Rn+index 的字数据→ Rd; 并将新地址 Rn+index→Rn
- LDR Rd, [Rn, Rm, LSL # 5] !** ; 内存地址为 Rn+Rm×32 的字数据→ Rd; 并将新地址 Rn+Rm×32→Rn
- LDR Rd, [Rn], Rm** ; 内存地址为 Rn 的字数据→ Rd; 并将新地址 Rn+Rm→Rn
- LDR Rd, [Rn], # index** ; 内存地址为 Rn 的字数据→ Rd; 并将新地址 Rn+index→Rn
- LDR Rd, [Rn], Rm, LSL # 5** ; 内存地址为 Rn 的字数据→ Rd; 并将新地址 Rn+Rm×32→Rn

Load/Store 指令举例

- LDR R1, [R0, #0x12]** ; 读出 R0+0x12 地址中的数据，保存到 R1
- LDR R1, [R0, -R2]** ; 中 (R0 的值不变); 将 R0-R2 地址处的数据读出，保存到 R1
- LDR R1, [R0]** ; 中 (R0 的值不变); 将 R0 地址处的数据读出，保存到 R1 中
- LDR R1, localdata** ; 加载一个字，该字位于变量 localdata 所; 在地址
- LDR R0, [R1], R2, LSL #2** ; 将地址为 R1 的内存单元数据读取到 R0; 中，然后 R1=R1+R2×4
- 在编程中常使用该指令将外设端口数据读到 R0 中：
- MOV R1, #UARTADD** ; 将外设端口地址 UART ADD 装入 R1 中
- LDR R0, [R1]** ; 将外设端口数据存到 R0 中

LDRT 用户模式的字数据加载指令

- 格式：**LDR{<cond>}T <Rd>, <addr>**
- 功能：同 LDR 指令。
- 说明：无论处理器处于何种模式，都将该指令当作一般用户模式下的内存操作。
- addr 所表示的有效地址必须是字对齐的，否则从内存中读出的数值需进行循环右移操作。

- **格式**：LDR{<cond>}BT <Rd>, <addr>
- **功能**：同 LDRB 指令。
- **说明**：无论处理器处于何种模式，都将该指令当作一般用户模式下的内存操作。

STR 字数据存储指令

- **格式**：STR{<cond>} <Rd>, <addr>
- **功能**：
 - (1) 寄存器 Rd 字数据 (32 位) → addr 地址中
 - (2) 还可以：合成的有效地址 → 基址寄存器
- **地址 addr**：简单的值、一个偏移量、被移位的偏移量。
- **寻址方式**同 LDR 指令。
- **例如**：
R1=0x40003000 R0=0x1A27E4F3
STR R0, [R1, #4] !
(1) 存储器地址 =R1+#4=0x40003000+#4=0x40003004
(2) R0→[0x40003004]= 0x1A27E4F3
(3) 修改 R1 R1+#4→ R1= 0x40003004

STR 指令应用举例 -2

- (2) 用 ARM 指令实现 $x = a \times (b + c)$

```
ADR    R4, b        ; 变量 b 处的地址装入 R4 中
LDR    R0, [R4]      ; 取 b
ADR    R4, c        ; 变量 c 处的地址装入 R4 中
LDR    R1, [R4]      ; 取 c
ADD    R2, R0, R1    ; R0+R1 → R2 , 即 b+c → R2
ADR    R4, a        ; 变量 a 处的地址装入 R4 中
LDR    R0, [R4]      ; 取 a
MUL    R2, R2, R0    ; R2×R0 → R2 , 即 (b+c)×a
        → R2
ADR    R4, x        ; 变量 x 处的地址装入 R4 中
STR    R2, [R4]      ; 存 x
```

- **格式**：LDR{<cond>}SB <Rd>, <addr>
- **功能**：同 LDRB 指令，但
 - (1) addr 地址内 8 位的字节数据 → Rd
 - (2) Rd 高 24 位按字节符号位扩展
- **例如**：
R1=0x10000000
[0x10000000]=0x93

LDRSB R0, [R1]

结果 R0=0xFFFFF93

STR 字数据存储指令举例

- STR R2, [R1, #16] ; 将 R2 的内容保存到 R1+16 为地址的内存中
- STR R0, [R7], # -8 ; 将 R0 的内容保存到 R7 中地址对应的内存 ; 中 , R7←R7-8
- STR R2, [R9, #consta-struct] ; consta-struct 是一个常量表达式 , ; 范围为 -4095 ~ 4095
- 在编程中常使用该指令将 R0 中的一个字 **存到外设端口寄存器**中：
- MOV R1, #UARTADD ; 将外设端口地址 UARTADD 装入 R1 中
- STR R0, [R1] ; 将数据保存到外设端口寄存器中

STRB 字节数据存储指令

- **格式**：STR{<cond>}B <Rd>, <addr>
- **功能**：Rd 低 8 位字节数据 → addr 内存地址
- **其他用法**同 STR 指令。
- **例如**：
假设 R1=0x40003000 R0=0x1A27E4F3

STRB R0, [R1, #4] !

(1) 存储器地址
=R1+#4=0x40003000+#4=0x40003004
(2) R0 低 8 位字节数据 → [0x40003004]= 0xF3
(3) 修改 R1 R1+#4 → R1= 0x40003004

- **格式**：LDR{<cond>}SH <Rd>, <addr>
- **功能**：同 LDRH 指令，但
 - (1) addr 地址内 16 位的半字数据 → Rd
 - (2) Rd 高 16 位按半字节符号位扩展
- **例如**：
R1=0x10000000
[0x10000000]=0x933D

LDRSH R0, [R1]

结果 R0=0xFFFF933D

STR 指令应用举例 -1

- (1) 用 ARM 指令实现 $x=(a+b)-c$

```
LDR    R4, =a        ; 变量 a 处的地址装入 R4 中
LDR    R0, [R4]      ; a → R0
LDR    R5, =b        ; 变量 b 处的地址装入 R5 中
LDR    R1, [R5]      ; b → R1
ADD    R3, R0, R1    ; R0+R1 → R3 , 即 a+b → R3
LDR    R4, =c        ; 变量 c 处的地址装入 R4 中
LDR    R2, [R4]      ; c → R2
SUB    R3, R3, R2    ; R3-R2 → R3 , 即 (a+b)-c
        → R3
LDR    R4, =x        ; 变量 x 处的地址装入 R4 中
STR    R3, [R4]      ; 存 x
```

STRH 半字数据存储指令

- **格式**：STR{<cond>}H <Rd>, <addr>
- **功能**：Rd 低 16 位半字数据 → addr 内存地址
- **说明**：addr 所表示的地址必须是半字对齐的。
- **其他用法**同 STR 指令。
- **例如**：
假设 R1=0x40003000 R0=0x1A27E4F3

STRH R0, [R1, #4] !

(1) 存储器地址
=R1+#4=0x40003000+#4=0x40003004
(2) R0 → [0x40003004]= 0xE4F3
(3) 修改 R1 R1+#4 → R1= 0x40003004

STRT 用户模式的字数据数据存储指令

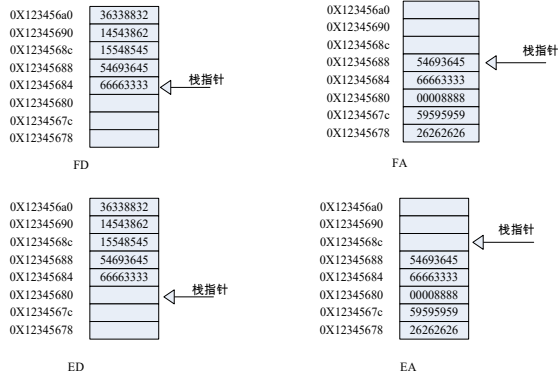
- 格式：STR{<cond>}T <Rd>, <addr>
- 功能：同 STR 指令。
- 说明：无论处理器处于何种模式，该指令都将被当作一般用户模式下的内存操作。

type 字段种类

- type 字段种类：

IA：每次传送后地址加
IB：每次传送前地址加
DA：每次传送后地址减
DB：每次传送前地址减
FD：满递减堆栈
ED：空递减堆栈
FA：满递增堆栈
EA：空递增堆栈

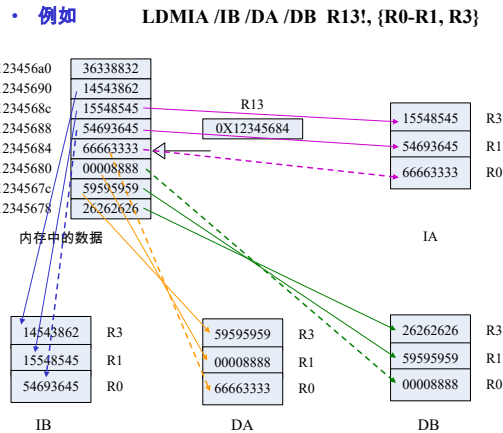
FD/ED/FA/EA 举例



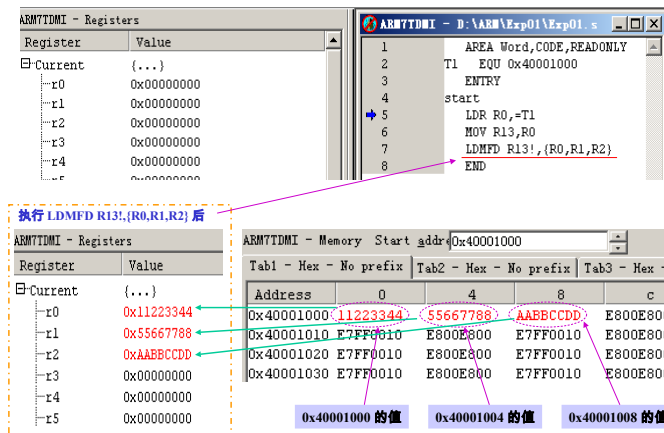
STRBT 用户模式的字节数据数据存储指令

- 格式：STR{<cond>}BT <Rd>, <addr>
- 功能：同 STRB 指令。
- 说明：无论处理器处于何种模式，该指令都将被当作一般用户模式下的内存操作。

IA/IB/DA/DB 举例



LDMFD 实验例 (出栈)



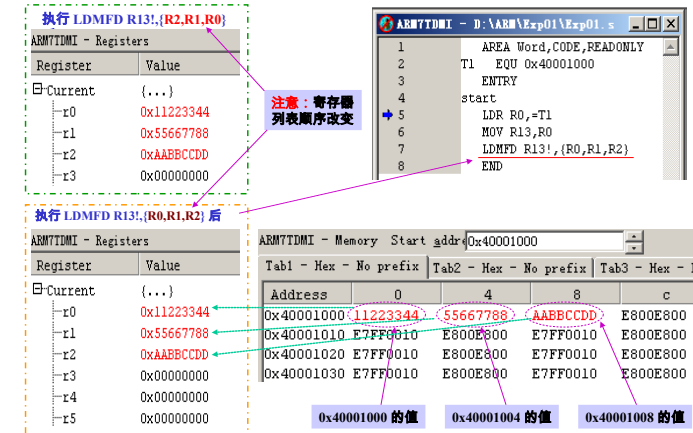
LDM 批量数据加载指令

- 格式：LDM{<cond>}{<type>} <Rn> {!, <regs> {^}}
- 功能：连续存储单元→寄存器（多个）
- 该指令一般用于多个寄存器数据的出栈。
- 格式说明：
- type 字段种类：8 种。
- Rn：基址寄存器，其值是内存单元的起始地址。不允许为 R15。
- Regs：寄存器列表，从最低序号寄存器开始，与书写顺序无关。
- ! 后缀：指令执行完毕后，将最后的地址写入基址寄存器。
- ^ 后缀：当 regs 中不包含 PC 时，该后缀用于指示指令所用的寄存器为用户模式下的寄存器。
- 否则，指示指令执行时，SPSR→CPSR。

FD/ED/FA/EA

- FD、ED、FA 和 EA 指定是满栈还是空栈，是升序栈还是降序栈，用于堆栈寻址。
- 满栈：栈指针指向上次写的最后一个数据单元。
- 空栈：栈指针指向第一个空闲单元。
- 降序栈在内存中反向增长；升序栈在内存中正向增长。
- ARM 微处理器支持四种类型的堆栈工作方式，即：
- (1) 满递增方式 FA (Full Ascending)：堆栈指针指向最后入栈的数据位置，且由低地址向高地址生成。
- (2) 满递减方式 FD (Full Decending)：堆栈指针指向最后入栈的数据位置，且由高地址向低地址生成。
- (3) 空递增方式 EA (Empty Ascending)：堆栈指针指向下一个入栈数据的空位置，且由低地址向高地址生成。
- (4) 空递减方式 ED (Empty Decending)：堆栈指针指向下一个入栈数据的空位置，且由高地址向低地址生成。

LDMFD 实验例 - 调整寄存器列表顺序



LDM 指令操作的伪代码

- LDM {<cond>} <addressing_mode> <Rd> {}, <registers>

指令操作的伪代码：

```
if ConditionPassed(cond) then
    addresss = start_address
    for i = 0 to 14
```

即 8 种 type 字段种类

；从 R0 开始检查并设置

```
    if registers_list[i] == 1 then
        Ri = Memory[addresss,4]
```

；广义修正

```
    if registers_list[15] == 1 then
```

；R15 是 PC

```
        value = Memory[addresss,4]
        if (architecture version 5 or above) then
            PC = value AND 0xFFFFFFF
```

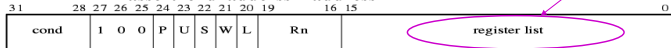
```
            T Bit = value[0]
```

```
        else
            PC = value AND 0xFFFFFFF
```

```
            addresss = address + 4
```

R15-R0 的序号

```
assert end addresss = address - 4
```



SWP 字数据交换指令

- 格式：SWP {<cond>} <Rd>, <op1>, [<op2>]

- 功能：Rd = [op2]

- [op2] = op1

- 从 op2 所表示的内存装载一个字并把这个字放置到目的寄存器 Rd 中，然后把寄存器 op1 的内容存储到同一内存地址中。

- op1、op2：均为寄存器。

- 例如：

- SWP R0, R1, [R2]；[R2] 字数据→R0，R1 字数据→[R2]

CDP 协处理器数据操作指令

- 格式：

- CDP {条件} 协处理器编码, 协处理器操作码 1, 目的寄存器, 源寄存器 1, 源寄存器 2, 协处理器操作码 2

- 功能：用于 ARM 处理器通知 ARM 协处理器执行特定的操作，若协处理器不能成功完成特定的操作，则产生未定义指令异常。其中协处理器操作码 1 和协处理器操作码 2 为协处理器将要执行的操作，目的寄存器和源寄存器均为协处理器的寄存器，指令不涉及 ARM 处理器的寄存器和存储器。

- 指令示例：

```
CDP P1, 2, C1, C2, C3
```

；命令 P1（1 号）协处理器，把自己的寄存器 C2 和寄存器 C3 作为操作数，进行第“2”方式的操作，结果放在寄存器 C1 中。

；即，P1：C2 操作 2 C3 → C1

STM 批量数据存储指令

- 格式：STM {<cond>} {<type>} <Rn> {}, <regs> {^}

- 功能：寄存器列表的值→连续的内存单元

- Rn：基址寄存器，其值为内存单元的起始地址为。

- Regs：寄存器列表。

- 该指令一般用于多个寄存器数据的入栈。

- 注意：最高序号的寄存器数据最先进栈。

- 其他参数用法同 LDM 指令。

- 例如：

- STMEA R13!, {R0-R12, PC}；将寄存器 R0-R12 以及程序计数器

；PC 的值保存到 R13 指示的堆

栈中

SWPB 字节数据交换指令

- 格式：SWP {<cond>} B <Rd>, <op1>, [<op2>]

- 功能：[op2] 一个字节→Rd 低 8 位，Rd 的高 24 位清 0
op1 低 8 位数据→[op2]

- 例如：

- SWPB R0, R1, [R2]；[R2] 一个字节数据→R0 低 8 位
；R1 低 8 位字节数据→[R2]

LDC 协处理器加载指令

- 格式：

LDC {条件} {L} 协处理器编码, 目的寄存器, [源寄存器]

- 功能：协处理器的目的寄存器←[ARM 的源寄存器] 间址存储器单元。

- 用于将源寄存器所指向的存储器中的字数据传送到目的寄存器中，若协处理器不能成功完成传送操作，则产生未定义指令异常。

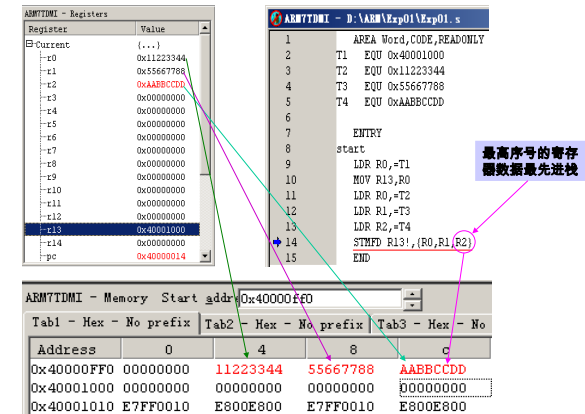
- {L} 选项：表示指令为长读取操作，如用于双精度数据的传输。

- 指令示例：

```
LDC P3, C4, [R5]
```

；将 ARM 处理器的寄存器 R5 所指向的存储器中的字数据传送到协处理器 P3 的寄存器 C4 中。

STM 入栈举例



3.2.6 ARM 协处理器指令

- ARM 处理器最多可支持 16 个协处理器，用于辅助 ARM 完成各种协处理操作。
- 在程序执行过程中，各协处理器只执行自身的协处理指令，而忽略属于 ARM 处理器和其他协处理器的指令。

- ARM 协处理器指令可分为 3 类：

(1) ARM 处理器用于初始化协处理器的数据操作指令 (CDP)。

(2) 协处理器寄存器和内存单元之间的数据传送指令 (LDC, STC)。

(3) ARM 处理器寄存器和协处理器寄存器之间的数据传送指令 (MCR, MRC)。

STC 协处理器存储指令

- 格式：

STC {条件} {L} 协处理器编码, 源寄存器, [目的寄存器]

- 功能：协处理器源寄存器字数据→[ARM 目的寄存器] 间址存储器单元。

- 用于将源寄存器中的字数据传送到目的寄存器所指向的存储器中，若协处理器不能成功完成传送操作，则产生未定义指令异常。

- {L} 选项：表示指令为长读取操作，如用于双精度数据的传输。

- 指令示例：

```
STCEQ P2, C4, [R5]
```

；当 Z=1 时，执行将协处理器 P2 的寄存器 C4 中的字数据传送到 ARM 处理器的寄存器 R5 所指向的存储器中。

MCR 和 MRC 指令

- 格式：
MCR/MRC {条件} 协处理器编码, 协处理器操作码 1, 目的寄存器, 源寄存器 1, 源寄存器 2, 协处理器操作码
- MCR 功能：协处理器寄存器 ← ARM 处理器寄存器。
- MRC 功能：ARM 处理器寄存器 ← 协处理器寄存器。
- ARM 处理器指定一个寄存器作为传送数据的源或接收数据的目标。
- 协处理器指定两个寄存器, 同时可以像 CDP 指令一样指定操作要求。
- 指令示例：

```
MCR    P2, 3, R2, C4, C5, 6
```

; 指定协处理器 P2 执行第 3 种操作, 类型 6, 操作数是 R2, 结果放在 C4 中。

```
MRC    P0, 3, R2, C4, C5, 6
```

; 指定协处理器 P0 执行第 3 种操作, 类型 6, 操作数 C4 和 C5, 结果送给 R2。

3.2.7 ARM 软件中断指令

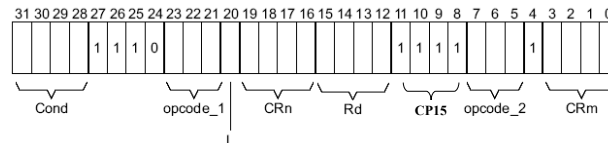
- ARM 处理器支持两条异常中断指令：
软件中断指令 SWI
断点中断指令 BKPT
- ARM 指令集中的软件中断指令是唯一一条不使用寄存器的 ARM 指令, 也是一条可以条件执行的指令。
- 因为 ARM 指令在用户模式中受到很大的局限, 有一些资源不能够访问。所以, 在需要访问这些资源时, 使用软件控制的唯一方法就是使用软件中断指令。

SWI 的参数传递

- 使用 SWI 指令时, 通常使用以下两种方法进行参数传递：
 - (1) 指令中使用 24 位的立即数 (非 0), 参数通过通用寄存器传递。此时, 24 位的立即数指定了用户请求的服务类型。
 - 例：
MOV R0, #34 ; 设置子功能号为 34, 作为入口参数
SWI 12 ; 调用 12 号软中断
 - (2) 指令中的 24 位立即数被忽略 (为 0), 用户请求的服务类型由寄存器 R0 的值决定, 参数通过其他的通用寄存器传递。
 - 例：
MOV R0, #12 ; 调用 12 号软中断
MOV R1, #34 ; 设置子功能号为 34
SWI 0 ; 软中断, 中断立即数为 0

ARM 920T 的协处理器

- ARM 920T 只有两个协处理器：CP14、CP15
- CP14：调试通信通道协处理器 DCC, 提供了两个 32bits 寄存器用于传送数据。
- CP15：系统控制协处理器, 配置和控制 caches、MMU、保护系统、时钟模式。有 16 个寄存器：CR0-CR15。
- CP15 寄存器只能被 MRC 和 MCR 指令访问。
- MRC/MCR {cond} coproc, opcode1, Rd, CRn, CRm {opcode2}
L 用于区分 MRC(L=1) 和 MCR(L=0)



SWI 软件中断指令

- 格式：SWI {<cond>} 24 位的立即数
- 功能：产生软件中断, 调用系统例程。
- 24 位的立即数：指定用户程序调用系统例程的类型, 其参数通过通用寄存器传递。
- 当 24 位的立即数被忽略时, 系统例程类型由寄存器 R0 指定, 其参数通过其他通用寄存器传递。
- 例如：
SWI 0X05 ; 调用编号为 05 的系统例程。

SWI 的操作如何获取立即数

- SWI 中断处理程序要通过读取 SWI 指令, 来获取 24 位立即数。
- 步骤：
 - (1) 确定引起中断的 SWI 指令是 ARM 指令还是 Thumb 指令, 可由 SPSR 得到。
 - (2) 取 SWI 指令的地址, 可由 LR 得到。
 - (3) 读出指令, 分解出立即数。
- 例：

```
T_bit EQU 0x20
SWI_Handler
STMFD SP!, {R0-R3, R12, LR} ; 现场保护
MRS R0, SPSR ; 读取 SPSR
TST R0, #T_bit ; 测试 T 标志位
LDRNEH R0, [LR, #-2] ; 若是 Thumb 指令, 读取 16 位指令码
BICNE R0, R0, #0xFF00 ; 对应位清 0, 取 Thumb 指令的 8 位立即数
LDREQ R0, [LR, #-4] ; 若是 ARM 指令, 读取 32 位指令码
BICEQ R0, R0, #0xFF000000 ; 对应位清 0, 取 ARM 指令的 24 位立即数
...
LDMFD SP!, {R0-R3, R12, PC}^ ; 恢复现场, 同时 SPSR ← CPSR
```

CP15 寄存器 CR0

- 寄存器 CR0 有 2 个, 具体被访问的寄存器由 Opcode_2 字段的值决定。
- ID 编码寄存器
该寄存器为只读寄存器, 返回 32 位的设备 ID 编码。通过读 CP15 寄存器 0 并将 opcode_2 字段设置 0 可以访问 ID 编码寄存器。例如：
MRC p15, 0, Rd, c0, c0, 0 ; 返回 ID 寄存器
- Cache 类型寄存器
该寄存器为只读寄存器, 它包含 ICache 和 DCache 的大小以及体系结构的相关信息。通过读 CP15 寄存器 0 并将 opcode_2 字段设置为 1 可以访问 Cache 类型寄存器, 例如：
MRC p15, 0, Rd, c0, c0, 1 ; 返回 Cache 的详细资料

SWI 指令操作的伪代码

- SWI {<cond>} <immed_24>
- 指令操作的伪代码：

```
if ConditionPassed(cond) then
    R14_svc = address of next instruction after the SWI instruction
    SPSR_svc = CPSR
    CPSR[4:0] = 0b10011 /* 进入管理模式 */
    CPSR[5] = 0 /* Execute in ARM state, T Bit = 0 */
    /* CPSR[6] is unchanged, fiq 不变 */
    CPSR[7] = 1 /* Disable normal interrupt, I Bit = 1, 禁止 irq */
    if high vectors configured then
        PC = 0xFFFFF008
    else
        PC = 0x00000008
```

异常类型	工作模式	特定地址 (低端)	特定地址 (高端)	优先级
复位	管理模式	0x00000000	0xFFFFF000	1
未定义指令	未定义指令中止模式	0x00000004	0xFFFFF004	6
软件中断 (SWI)	管理模式	0x00000008	0xFFFFF008	6

BKPT 断点中断指令

- 格式：BKPT 16 位的立即数
- 功能：产生软件断点中断, 用于软件调试。
- 16 位的立即数用于保存软件调试中额外的断点信息。
- 例：
BKPT
BKPT 0xF02C

3.2.8 程序状态寄存器指令

- **作用**：状态寄存器和通用寄存器间传送数据。
- 总共有两条指令：MRS 和 MSR。
- 两者结合可用来修改程序状态寄存器的值。

3.3 Thumb 指令集

- Thumb 指令集可以认为是 ARM 指令集的子集。
- Thumb 指令长度为 16 位，但其数据处理指令的操作数仍然是 32 位的，指令寻址地址也是 32 位的。
- 由于是压缩的指令，因此，在 ARM 指令流水线中实现 Thumb 指令时，**先动态解压缩**，然后作为标准的 ARM 指令来执行。
- 如何区分指令取决于 CPSR 中的第五位：**T**（T=0 为 ARM 状态，T=1 为 Thumb 状态）。
- **Thumb 指令集中没有**：乘法指令、64 位乘法指令、协处理器指令、数据交换指令、程序状态寄存器指令，而且指令的第二操作数受到限制，除了**跳转指令 B 有条件执行功能**外，其他指令均为无条件执行。
- Thumb 指令系统不是完整的指令体系，必须与 ARM 配合使用。
- **Thumb 指令集有 4 大类**：
 - 数据处理指令
 - 跳转指令
 - Load/Store 指令
 - 软件中断指令

数据处理指令 2

格 式	功 能
AND Rd,Rn	Rd=Rd&Rn；Rd、Rn 为 R0 ~ R7
ORR Rd,Rn	Rd=Rd Rn；Rd、Rn 为 R0 ~ R7
EOR Rd,Rn	Rd=Rd^Rn；Rd、Rn 为 R0 ~ R7
BIC Rd,Rn	Rd=Rd&(~Rn)；Rd、Rn 为 R0 ~ R7
ASR Rd,Rn	Rd=Rd 算术右移 Rn 位；Rd、Rn 为 R0 ~ R7
ASR Rd,Rn,imm_5	Rd=Rn 算术右移 imm_5 位；Rd、Rn 为 R0 ~ R7，imm_5 为 1 ~ 32 之间的数值
LSL Rd,Rn	Rd=Rd 逻辑左移 Rn 位；Rd、Rn 为 R0 ~ R7
LSL Rd,Rn,imm_5	Rd=Rn 逻辑左移 imm_5 位；Rd、Rn 为 R0 ~ R7
LSR Rd,Rn	Rd=Rd 逻辑右移 Rn 位；Rd、Rn 为 R0 ~ R7
LSR Rd,Rn,imm_5	Rd=Rn 逻辑右移 imm_5 位；Rd、Rn 为 R0 ~ R7
ROR Rd,Rn	Rd=Rd 循环右移 Rn 位；Rd、Rn 为 R0 ~ R7
CMP Rn,Rm	根据 Rn-Rm 的值，修改 CPSR 的状态标志位；Rn、Rm 为 R0 ~ R7
CMN Rn,imm_8	根据 Rn-imm_8 的值，修改 CPSR 的状态标志位；Rn 为 R0 ~ R7
CMN Rn,Rm	根据 Rn+Rm 的值，修改 CPSR 的状态标志位；Rn、Rm 为 R0 ~ R7
TST Rn,Rm	根据 Rn&Rm 的值，修改 CPSR 的状态标志位；Rn、Rm 为 R0 ~ R7

MRS 程序状态寄存器到通用寄存器的数据传送指令

- **格式**：MRS{<cond>} <Rd>, CPSR/SPSR
- **功能**：CPSR/SPSR→Rd
- 用于以下场合保存状态：
 - 进入中断服务程序
 - 进程切换当
- **例如**：
- MRS R0, CPSR ；状态寄存器 CPSR→R0

Thumb 指令集与 ARM 指令集区别

- Thumb 指令集与 ARM 指令集在以下几个方面有**区别**：
- **跳转指令**。条件跳转在范围上有更多的限制，转向子程序**只具有无条件转移**。
- **数据处理指令**。对通用寄存器进行操作，操作结果需放入其中一个操作数寄存器，**没有第三个寄存器**。
- **单寄存器加载和存储指令**。Thumb 状态下，单寄存器加载和存储指令**只能访问寄存器 R0 ~ R7**。
- **批量寄存器加载和存储指令**。LDM 和 STM 指令可以将任何范围为 R0 ~ R7 的寄存器子集加载或存储，**PUSH 和 POP 指令使用堆栈指针 R13** 作为基址实现满递减堆栈，除 R0 ~ R7 外，PUSH 指令还可以存储链接寄存器 R14，并且 POP 指令可以加载程序指令 PC。

跳转指令

格 式	功 能
B{<cond>} label	PC=label 若有 cond，则 label 必须在当前指令的 -256 ~ +256 字节范围内； 否则，label 必须在当前指令的 -2KB ~ +2KB 范围内
BL label	R14=PC+4，PC=label label 必须在当前指令的 -4MB ~ +4MB 范围内
BX Rn	PC=Rn，且切换处理器状态

MSR 通用寄存器到程序状态寄存器的数据传送指令

- **格式**：MSR{<cond>} CPSR/SPSR_<field>,<op1>
- **功能**：CPSR/SPSR_field ← op1
- **op1**：通用寄存器、立即数
- 用于以下场合恢复状态：
 - 退出中断服务程序
 - 进程切换
- **例如**：
 - MSR CPRS_f, R0 ；R0→CPRS (只修改条件域)
 - MRS CPSR_fsxc, #5 ；#5→CPRS

状态寄存器位	Field 域	标识
位 [31:24]	条件标志域	f
位 [23:16]	状态位域	s
位 [15:8]	扩展位域	x
位 [7:0]	控制位域	c

数据处理指令 1

格 式	功 能
MOV Rd,imm_8	Rd=imm_8；Rd 为 R0 ~ R7，imm_8 为 8 位立即数
MOV Rd,Rn	Rd=Rn；Rd、Rn 为 R0 ~ R15
MOVN Rd,Rn	Rd=~Rn；Rd、Rn 为 R0 ~ R7
NEG Rd,Rn	Rd=-Rn；Rd、Rn 为 R0 ~ R7
ADD Rd,Rn,imm	Rd=Rn+imm；Rd 为 R0 ~ R7，Rn 为 R0 ~ R7 或 PC 或 SP。Rn 为 PC 或 SP 时，imm 为 10 位立即数；否则，imm 为 3 位立即数
ADD Rd,Rn,Rm	Rd=Rn+Rm；Rd、Rn、Rm 为 R0 ~ R7
ADD Rd,imm	Rd=Rd+imm；Rd 为 R0 ~ R7 或 SP Rd 为 SP 时，imm 为 -508 ~ +508 间的 4 整数倍的数；否则，imm 为 8 位立即数
ADD Rd,Rn	Rd=Rd+Rn；Rd、Rn 为 R0 ~ R15
ADC Rd,Rn	Rd=Rd+Rn+carry；Rd、Rn 为 R0 ~ R7，carry 为进位标志值
SUB Rd,Rn,imm_3	Rd=Rn-imm_3；Rd、Rn 为 R0 ~ R7，imm_3 为 3 位立即数
SUB Rd,Rn,Rm	Rd=Rn-Rm；Rd、Rn、Rm 为 R0 ~ R7，
SUB Rd,imm	Rd=Rd-imm；Rd 为 R0 ~ R7 或 SP Rd 为 SP 时，imm 为 -508 ~ +508 间的 4 整数倍的数；否则，imm 为 8 位立即数
SBC Rd,Rn	Rd=Rd-Rn-!carry；Rd、Rn 为 R0 ~ R7，carry 为进位标志值
MUL Rd,Rn	Rd=Rd×Rn；Rd、Rn 为 R0 ~ R7

Load/Store 指令

格 式	功 能
LDR Rd,[Rn,imm]	Rd=地址 (Rn+imm) 中的字数据；Rd 为 R0 ~ R7，Rn 为 R0 ~ R7 或 SP 或 PC；若 Rn 为 PC 或 SP，imm 为 5 位立即数，否则 imm 为 8 位立即数
LDR Rd,[Rn,Rm]	Rd=地址 (Rn+Rm) 中的字数据；Rd、Rn、Rm 为 R0 ~ R7
LDRH Rd,[Rn,imm_5]	Rd=地址 (Rn+imm_5) 中的无符号半字数据；Rd、Rn 为 R0 ~ R7，imm_5 为 5 位立即数
LDRH Rd,[Rn,Rm]	Rd=地址 (Rn+Rm) 中的无符号半字数据；Rd、Rn、Rm 为 R0 ~ R7
LDRB Rd,[Rn,imm_5]	Rd=地址 (Rn+imm_5) 中的无符号字节数据；Rd、Rn 为 R0 ~ R7
LDRB Rd,[Rn,Rm]	Rd=地址 (Rn+Rm) 中的无符号字节数据；Rd、Rn、Rm 为 R0 ~ R7
LDRSH Rd,[Rn,Rm]	Rd=地址 (Rn+Rm) 中的有符号半字数据；Rd、Rn、Rm 为 R0 ~ R7
LDRSB Rd,[Rn,Rm]	Rd=地址 (Rn+Rm) 中的有符号字节数据；Rd、Rn、Rm 为 R0 ~ R7
STR Rd,[Rn,imm]	Rd=地址 (Rn+imm) 中的字数据；Rd 为 R0 ~ R7，Rn 为 R0 ~ R7 或 SP 或 PC；若 Rn 为 PC 或 SP，imm 为 5 位立即数，否则 imm 为 8 位立即数
STR Rd,[Rn,Rm]	地址 (Rn+Rm) 处的字数据=Rd；Rd、Rn、Rm 为 R0 ~ R7
STRH Rd,[Rn,imm_5]	地址 (Rn+imm_5) 处的无符号半字数据=Rd；Rd、Rn 为 R0 ~ R7
STRH Rd,[Rn,Rm]	地址 (Rn+Rm) 处的无符号半字数据=Rd；Rd、Rn、Rm 为 R0 ~ R7
STRB Rd,[Rn,imm_5]	地址 (Rn+imm_5) 处的无符号字节数据=Rd；Rd、Rn 为 R0 ~ R7
STRB Rd,[Rn,Rm]	地址 (Rn+Rm) 处的无符号字节数据=Rd；Rd、Rn、Rm 为 R0 ~ R7
LDMIA Rd![],regs	regs 以 Rd 为起始地址的连续字数据；regs 为寄存器列表
STMIA Rd![],regs	以 Rd 为起始地址的连续字数据=regs；regs 为寄存器列表
PUSH regs,[LR]	[SP...]=regs!, LR；LR 即 R14，SP 即 R13
POP regs,[PC]	regs!, PC=SP, LR=PC, R15, SP 即 R13

第3章 结 束

格 式	功 能
SWI 8 位立即数	8 位立即数为中断号